

Mini Project — Running LLMs in Different Environments

Run AI models on your own computer, in Google Colab, and on a cloud server, then compare what you find.

100 points • no prior experience with these tools needed

1. What this project is about

Large language models (LLMs), the kind of AI behind ChatGPT, can also run on ordinary computers, free of charge. Models such as Llama, Qwen, Gemma, and Mistral can be downloaded and run by anyone. You will run several of them yourself, in three different places, and compare how they behave.

Treat this as an experiment rather than a test. Your goal is not to make everything work perfectly. If a model will not run because your computer runs out of memory, that is a real and useful result: write down what happened, what you tried, and what you learned. Problems that you investigate and explain earn full credit.

2. Words you will see

Term	What it means
LLM	Large language model — AI that reads and writes text
Hosting	Running a model yourself, on your own computer or a server
GPU	A graphics card. AI models run much faster on one than on a normal processor (CPU)
Model size (3B, 7B, 14B)	How large a model is. Bigger ones usually answer better but need more memory
Quantization	A compressed version of a model — smaller and faster, slightly less accurate
API	A way for your own program to talk to a model, instead of typing into a chat window
Temperature	A setting that controls answers: low is predictable, high is more creative and less reliable

3. Using AI assistants

You are not expected to know how any of this works beforehand, and this document does not give installation steps on purpose. Use an AI assistant such as ChatGPT, Claude, or Copilot for the technical detail. You can ask it for step-by-step instructions for your own operating system, paste these requirements into it and ask for a plan, show it an error message and ask what it means, or give it the numbers you collected and ask for help organizing them into tables and charts. Documentation, videos, and forums are equally welcome.

What an assistant cannot do for you is the part that is actually being graded. This is a project rather than a written assignment because the point is to show that you personally installed the tools, ran the models, watched them succeed and fail, took your own measurements, and can talk through the process afterwards. Your report and your video are the evidence of that, so make sure you can explain every step you show.

4. Suggested order

Work through the environments in this order; each is easier once you have done the one before it.

Step	Focus	You are done when...
Start	Your own computer	one model answers all five of your prompts on your machine
Next	Google Colab	the same prompts run on a free cloud GPU, and you have your first timing numbers
Last	Jetstream2, then the write-up	you have tried a larger model on the server and finished the write-up

5. Before you begin: make a plan

You need a computer you can install software on, a Google account for Colab, and access to Jetstream2 if it is provided to the class.

Write your five prompts before you install anything. You will send these same five questions to every model, in every environment. At least one should be a cybersecurity question, and at least one should be something you expect the model to answer badly.

Decide what data you will collect before you collect it. Draw the empty table you intend to fill in: model name, model size, where it ran, how long it took to load, how long it took to answer, how fast it produced text, and how much memory it used. Fill in the same table in every environment so the rows can be compared. Include this plan in your report, with anything you changed about it later and why.

This matters because if your questions or your measurements change between experiments, you cannot tell whether a difference came from the model, the hardware, or the way you tested it. Deciding both in advance is what makes your comparison meaningful.

6. The three environments

Your own computer. You install and run the model yourself. It is private, stays installed, and is fully under your control, but it is limited by the hardware you own, and most laptops have no AI-capable GPU, so answers come slowly. That is expected, and is one of the results you will report. If your computer does have a GPU, use it and report both.

Google Colab, free tier. You run models in a browser on Google's computers, including a free T4 GPU. Nothing is installed on your machine and you can start in minutes. Sessions expire, though, and whatever you saved there is usually gone when they do.

Jetstream2 virtual machine. A cloud Linux server with a powerful GPU, reached by SSH or a web desktop, like the professional cloud services companies pay for. It stays as you left it, so it is the best place to try larger models. It needs a little system administration, and it uses the class's shared allocation, so shut it down when you finish.

If Jetstream2 is not available, the tasks that need it are not required and are not graded, and those points are spread across the rest of the project. Your instructor will tell you which applies.

7. Required tasks

Each task lists what to do and the evidence to include in your report. Screenshots should be readable and show the command or window that produced the result. Points are listed in §10.

Task 1 — Try different hosting tools

Use at least two different tools for running models, for example Ollama, LM Studio, Open WebUI, llama.cpp, or Hugging Face Transformers. At least one of them must have a graphical or web interface.

Evidence: screenshots of each tool running, and an answer generated in each environment.

Task 2 — Try different models

Run at least three models from at least two different families, for example Llama, Qwen, Gemma, Phi, or Mistral. Include at least two different sizes, and try at least one model downloaded from Hugging Face rather than from a tool's built-in list.

Evidence: the model names and sizes, and each model's answers to your five prompts.

Task 3 — Interact in different ways

Use at least two ways of talking to a model: a command line, a Python script, a direct API call, or a graphical interface. Use the same prompt where you can, so the comparison is fair.

Evidence: each method shown working.

Task 4 — Change the settings

Change the temperature, trying a low value and a high one on the same prompt and model, and change one more setting of your choice, such as top-p, maximum tokens, or context length.

Evidence: the answers at each setting, and a few sentences on what changed and whether it would matter for security work.

Task 5 — Collect your performance data

Fill in the table you planned in §5. For each environment, record the hardware: processor, memory, and GPU if there is one. For your runs, measure how long the model took to load, how long it took to answer, how fast it generated text if your tool reports that, and how much memory it used. Measure the same way every time and say how you did it.

Evidence: your completed data table, plus the output of whatever command you used to check the hardware.

8. Final analysis

Build tables and at least one chart comparing the three environments on ease of setup, speed, the model sizes that ran successfully (and those that did not), cost, privacy, and reliability. Then recommend the best environment for each of these three situations, using your own numbers as the reason:

- everyday work for a security analyst;
- work involving sensitive or confidential data;
- a quick experiment that has to be finished today.

9. What to hand in

Submit this

one PDF report containing all of your writing, tables, charts, and screenshots, with the link to your video on the first page. Nothing else is submitted separately.

Organize the report by the tasks above and label each screenshot with the task number. Include your data plan from §5, and the problems you ran into and how you handled them. A report in which nothing went wrong reads as work that was never really tested.

For the video, record your screen for five to ten minutes and talk through it. Show one model actually generating text in each environment, at least one graphical interface, and your comparison. Say what surprised you and what you would do differently. Use live screens rather than slides of screenshots.

10. Grading

Required components — 70 points. The things you have to build and run.

Component	What it includes	Points
Three environments working	your computer; Google Colab; the Jetstream2 server	15
Hosting tools (<i>Task 1</i>)	two tools, at least one with a graphical interface	12
Models (<i>Task 2</i>)	three models; two families; two sizes; one from Hugging Face	15
Ways of interacting (<i>Task 3</i>)	at least two of: command line, script, API call, graphical interface	8
Settings (<i>Task 4</i>)	temperature high and low; one further setting	8
Data plan and measurements (§5, <i>Task 5</i>)	the plan written in advance; hardware and timings recorded the same way everywhere	12

Presenting your work — 30 points.

Deliverable	What it includes	Points
Report	comparison tables and at least one chart; the three recommendations, each backed by your own numbers; clear structure and labelled screenshots	15
Video	five to ten minutes of live screens: a model running in each environment, and your comparison	15

Use your own measurements, and keep confidential data and passwords out of every model and screenshot.